University College Dublin — School of Computer Science

COMP41720: Distributed Systems (2026-27) | Dr. Avishek Nag

Lab 2: Sagas, the Outbox Pattern, and Eventual Consistency

Week 6 | Labs Portfolio: 10% of the module, pass/fail | Duration: Two 2-hour sessions | Assessed in the session, in pairs

Learning Outcomes

- Implement the saga pattern (orchestration or choreography) to coordinate a multi-service workflow without distributed ACID transactions
- Implement the transactional outbox pattern to solve the dual-write problem and publish events reliably
- Design compensating actions and demonstrate them under injected failure
- Build idempotent consumers that tolerate at-least-once (duplicate) delivery
- Reason about consistency models — identify the model a system actually implements and judge where it is appropriate

Prerequisites

- Lab 1 completed — this lab reuses your RabbitMQ setup and producer/consumer skills
- Java (Spring Boot) or Python (FastAPI) development environment; Docker and Docker Compose
- Module text Ch. 7 — *Consistency, Transactions, and Sagas* (assigned reading for Week 6)

Background

In Week 6 we saw why ACID transactions do not span service boundaries: locking across networks couples availability to the slowest participant. The saga pattern replaces one distributed transaction with a sequence of local transactions, each with a **compensating action** that semantically undoes it if a later step fails. The transactional outbox solves the companion problem — the dual write — where a service must update its own database and publish an event, and a crash between the two silently corrupts the system.

Your scenario is an AI-native one, adapted from the module text's running case study: a **document-ingestion pipeline**. When a tenant uploads a document, three things must happen across three services: their usage quota is reserved, the document is stored, and its embeddings are generated for search. If embedding fails, the quota must be released and the document marked failed — the system must converge to a sensible state, not strand a half-ingested document against a charged quota. That is a saga.

Setup

Step 1: Reuse your Lab 1 RabbitMQ

```
docker run -d -p 5672:5672 -p 15672:15672 rabbitmq:4-management
```

Management UI at http://localhost:15672 (guest/guest).

Step 2: Scaffold three small services

Create three services (Spring Boot or FastAPI). To keep the automated test harness happy, use these fixed ports and paths:

- **Quota Service** — port 8081: POST /quota/reserve, POST /quota/release
- **Document Service** — port 8082: POST /documents (entry point), GET /documents/{id}/status
- **Embedding Service** — port 8083: consumes events; env var FAILURE_RATE (0.0–1.0) injects random failure

Step 3: Embeddings without an API dependency

The embedding step may call a local model via Ollama if you wish, but a simulated embedder (hash the text into a fixed-size vector, sleep 200 ms) is fully acceptable — this lab assesses the coordination pattern, not the model. Do not make your saga depend on a paid cloud API.

Part 1: Guided Exercises (UNASSESSED — ~60 min)

Exercise 2.1: Break something — the dual-write problem

- In the Document Service, write the document record and then publish a document.stored event as two separate operations, with a deliberate crash (System.exit / os._exit) between them behind a flag.
- Trigger the crash. Observe: the database has the document, the queue never got the event — the embedding pipeline will never run. This silent divergence is the dual-write problem.

Exercise 2.2: Fix it — a minimal transactional outbox

- Add an outbox table (or JSON file) written in the same local transaction as the document record. A relay poller reads unpublished rows and publishes them to RabbitMQ, marking them published.
- Repeat the crash test. On restart, the relay finds the unpublished row and delivers the event — nothing is lost.
- Now crash the relay after publishing but before marking published. Observe the duplicate delivery on restart. Conclusion: the outbox gives at-least-once delivery, so consumers must be idempotent.

Exercise 2.3: Watch eventual consistency happen

- Add a search projection that consumes document.embedded events into a separate read store. Measure the lag between POST /documents returning and the document appearing in the projection. That interval is your inconsistency window — you will measure it formally in Task B.

Part 2: Assessed Work (ASSESSED — ~90 min)

Task A: The Document-Ingestion Saga

- **Happy path:** POST /documents → quota reserved → document stored (via outbox) → embeddings generated → status COMPLETED. Events flow through RabbitMQ; every event carries a correlation ID (the document ID).
- **Compensation path:** with FAILURE_RATE=1.0 on the Embedding Service, the saga must release the reserved quota, mark the document FAILED, and settle with no stranded state.
- **Coordination style:** choose orchestration (a saga coordinator) or choreography (services react to each other's events). Either is acceptable — your ADR must justify the choice.
- **Idempotent consumers:** every consumer deduplicates by message ID — redelivering any event must not double-reserve quota or double-index a document.
- **Status endpoint:** GET /documents/{id}/status returns PENDING, COMPLETED, or COMPENSATED/FAILED — the saga's state must be observable.

Task B: Consistency Verification

- **Test 1 — compensation under failure:** run with FAILURE_RATE=1.0; show (logs + RabbitMQ UI screenshots) the compensating actions firing and the final settled state.
- **Test 2 — duplicate delivery:** redeliver a consumed event manually (or crash the relay as in Exercise 2.2); show your idempotency guard rejecting the duplicate.
- **Test 3 — the inconsistency window:** with FAILURE_RATE=0.0, measure the time from POST /documents (202 Accepted) to the projection showing the document, across at least 20 requests; report min/median/max.

- **Written analysis (~half page):** what anomalies can a client observe mid-saga (document visible but unsearchable; quota charged for a failed document before compensation lands)? Which mitigation would you deploy first, and what does it cost?

Task C: Documentation

- **(Appendix A) — including this lab's required question: state the consistency model your AI-generated code actually implements. Would it be appropriate for a banking ledger? For a model registry serving inference? Why or why not?**
- **One ADR (Appendix A): orchestration vs. choreography for this saga — or outbox vs. event sourcing, if you prefer. Context, decision, consequences, alternatives.**
- **Sequence diagram** (draw.io or hand-drawn): both the happy path and the compensation path, with every event named.
- **Reflection (300–500 words):** where is the line between the consistency your system guarantees and the consistency your status endpoint merely reports? If this pipeline ran at 100× load, which component breaks first?

AI Tool Integration

AI tools (Copilot, ChatGPT, Claude, or your preferred assistant) are encouraged for scaffolding services, RabbitMQ configuration, and debugging. AI assistants reliably produce saga code that **looks right and compensates wrong** — typical misses: non-idempotent consumers, compensation that assumes the forward action committed, outbox relays that mark-then-publish. Finding these is the point of the reflection. Verify everything under the Task B failure tests.

Important: do not blindly copy AI-generated code. Understand what you submit — Lab 2 material is in scope for the supervised lab exam in Week 10, where no AI assistants are available, and your project viva may probe any pattern used here.

Appendix A: Architectural Decision Record (ADR) Template

```
ARCHITECTURAL DECISION RECORD (ADR)
-----------------------------------
Title: [Short descriptive title]
Date: [Date]
Status: [Proposed / Accepted / Deprecated]
Context: [The situation and the forces at play]
Decision: [The choice that was made]
Consequences: [Positive consequences | Negative consequences]
Alternatives considered: [Other options and why they were not chosen]
```

Assessment — in the session, pass/fail

There is no Brightspace submission for this lab. During the session a demonstrator ticks three criteria at your machine:

1. The harness is green: the happy path completes across Quota (:8081), Document (:8082) and Embedding (:8083).

2. The compensation path runs: with FAILURE_RATE raised, a failed step triggers its compensating actions and leaves no orphaned state.

3. You can state your own inconsistency-window numbers from a run you did in the room, not from the brief.

Three ticks and this lab is passed. You work in pairs, and both of you must be able to explain any part of it on request.

The Labs Portfolio is 10% of the module and is pass/fail. You pass it by being ticked on at least three of the four labs and by submitting one ADR.

The one ADR — due Week 11

Across the whole trimester you submit a single Architectural Decision Record of at most 600 words, on whichever of the four labs you choose, by Friday of Week 11 on Brightspace. Use the template in Appendix A. It is judged competent or not yet competent against Appendix C of the module text; generic, uncontextualised reasoning is not competent.

Keep your code and evidence

Nothing is uploaded for this lab, but keep your repository, your README and any screenshots. Lab 2 material is in scope for the supervised lab exam in Week 10 — 90 minutes, individual, no AI assistants, drawn from Labs 1-3.